



Multi-Agent Research System - Detailed Project Prompt

Project Overview

Build a production-ready multi-agent AI system where specialized AI agents collaborate to conduct comprehensive research on complex topics, synthesize findings, and produce high-quality reports with citations and evaluation metrics.

▮ Project Objectives

Primary Goals

1. **Demonstrate Agentic AI Architecture** - Move beyond simple RAG to agents that plan, reason, and execute multi-step workflows
2. **Show Production Engineering Skills** - State management, error handling, streaming responses, deployment
3. **Implement Evaluation Framework** - Measure quality using RAG Triad and custom metrics
4. **Prove Systems Thinking** - Orchestrate multiple agents working collaboratively toward complex goals

Learning Outcomes

- Master LangGraph for stateful agent workflows
- Implement tool-calling patterns (web search, code execution, database queries)
- Build evaluation systems for non-deterministic LLM outputs
- Deploy multi-agent systems with FastAPI and streaming
- Handle real-world challenges: hallucinations, infinite loops, error recovery

▮ Functional Requirements

User Story

As a researcher/analyst, I want to ask complex questions requiring multi-source investigation, **so that** I receive comprehensive, well-researched reports with citations and quality metrics, **saving me hours of manual research**.

Core Functionality

1. Query Processing

- **Input:** Natural language research question
- **Examples:**
 - "Analyze the 2026 impact of Indonesia's new EV policies on the automotive industry and consumer adoption"
 - "Compare carbon footprint reduction strategies in palm oil production across Malaysia and Indonesia"
 - "What are the most promising applications of GraphRAG in enterprise knowledge management?"

2. Multi-Agent Collaboration

Required Agents (minimum 4):

Planner Agent:

- Decomposes complex queries into research sub-tasks
- Creates execution plan with dependencies
- Assigns tasks to appropriate specialist agents
- **Output:** Structured task list with priorities

Research Agent:

- Conducts web searches using APIs (Tavily, Serper)
- Queries internal knowledge bases (vector DB)
- Extracts relevant information from sources
- **Tools:** Web search, document retrieval, URL scraping
- **Output:** Raw research findings with source URLs

Data Analyst Agent:

- Processes quantitative data
- Performs calculations and statistical analysis
- Creates data summaries and visualizations
- **Tools:** Python REPL, pandas, numpy

- **Output:** Data insights, charts, statistical findings

Synthesis Agent (Writer):

- Combines findings from Research and Analyst agents
- Generates coherent narrative report
- Ensures proper citations and structure
- **Output:** Draft report with citations

Critic Agent (Quality Reviewer):

- Reviews output for hallucinations
- Checks factual accuracy against sources
- Identifies logical inconsistencies
- Validates citations
- **Decision:** Accept or send back for revision
- **Output:** Quality assessment + revision requests

3. State Management & Memory

- **Conversation State:** Persist multi-turn interactions
- **Agent Memory:** Track what each agent has done
- **Checkpointing:** Save/resume workflow at any point
- **Shared Context:** All agents access common knowledge

Implementation: Use LangGraph's `Checkpointers` with PostgreSQL or Redis backend

4. Tool Integration

Required Tools:

- **Web Search:** Tavily API or Serper API for real-time information
- **Python REPL:** Safe code execution for data analysis
- **Document Retrieval:** Vector database query (Pinecone, Weaviate, Chroma)
- **Web Scraper:** Custom tool for extracting content from URLs

Optional Advanced Tools:

- **Calculator:** Precise numerical computations
- **SQL Query:** Database access for structured data
- **API Clients:** Domain-specific APIs (financial data, weather, etc.)

5. Evaluation & Quality Metrics

RAG Triad Metrics (using TruLens or Ragas):

1. **Context Relevance:** Are retrieved documents relevant to query?
 - Target: >0.80
 - Measures retrieval quality
2. **Groundedness:** Is the answer supported by retrieved context?
 - Target: >0.90
 - Measures hallucination rate
3. **Answer Relevance:** Does answer actually address the question?
 - Target: >0.85
 - Measures response quality

Custom Metrics:

- **Task Success Rate:** % of queries successfully completed
- **Citation Accuracy:** % of claims with valid sources
- **Agent Collaboration Efficiency:** Number of agent interactions per task
- **Token Usage:** Cost per query
- **Latency:** Time to complete research (target: <2 minutes)

6. Output Format

Research Report Structure:

```
# [Research Question]

## Executive Summary
[Key findings in 2-3 sentences]

## Methodology
- Sources consulted: [N web sources, M database entries]
- Analysis performed: [Statistical methods, comparisons]
- Agents involved: [Which agents contributed]

## Findings

### Topic 1
[Detailed findings with citations]
Source: [URL or document reference]

### Topic 2
[More findings]

## Data Analysis
[If applicable: charts, tables, statistical results]
```

```
## Conclusions
[Synthesis and implications]

## Quality Metrics
- Context Relevance: 0.87
- Groundedness: 0.92
- Answer Relevance: 0.89
- Sources Verified: 12/13 (92%)

## References
1. [Author/Source]. [Title]. [URL]. Retrieved: [Date]
2. ...
```

▮ Technical Architecture

System Components

```
graph TD
    User[User Query] --> Router[Query Router]
    Router --> Planner[Planner Agent]
    Planner --> Research[Research Agent]
    Planner --> Analyst[Data Analyst Agent]

    Research --> Tools1[Web Search API]
    Research --> Tools2[Vector DB]
    Research --> Tools3[Web Scraper]

    Analyst --> Tools4[Python REPL]
    Analyst --> Tools5[Data Processing]

    Research --> Synthesis[Synthesis Agent]
    Analyst --> Synthesis

    Synthesis --> Critic[Critic Agent]
    Critic -->|Revision Needed| Synthesis
    Critic -->|Approved| Output[Final Report]

    Output --> Eval[Evaluation System]
    Eval --> Metrics[Quality Metrics]

    StateDB[(PostgreSQL/Redis)] -.->|State Persistence| Planner
    StateDB -.-> Research
    StateDB -.-> Analyst
    StateDB -.-> Synthesis
```

Tech Stack

Core Framework:

- **Python:** 3.10+
- **LangGraph:** Stateful agent orchestration (primary framework)

- **LangChain:** Tool integration, LLM abstractions

Alternative Frameworks (choose one):

- **CrewAI:** Role-based multi-agent (simpler, less control)
- **AutoGen:** Conversational agents (Microsoft)

Recommendation: Use **LangGraph** for maximum control and production readiness

LLM Providers:

- **Primary:** OpenAI GPT-4 or Anthropic Claude 3.5 Sonnet
- **Fallback:** OpenAI GPT-4o-mini for cost optimization
- **Budget Option:** Open-source (Llama 3, Mistral) via Groq/Together AI

Tools & APIs:

- **Web Search:** Tavily API (recommended) or Serper API
- **Vector Database:** Pinecone (managed), Weaviate (self-hosted), or Chroma (embedded)
- **Embeddings:** OpenAI `text-embedding-3-small` or Sentence Transformers
- **Code Execution:** `PythonREPL` from LangChain (sandboxed)

State Management:

- **Database:** PostgreSQL or Redis
- **ORM:** SQLAlchemy (for PostgreSQL)
- **Checkpointing:** LangGraph's built-in checkpointing

Evaluation:

- **Framework:** TruLens (recommended) or Ragas
- **Custom Metrics:** Manual implementation

API & Deployment:

- **API Framework:** FastAPI with WebSocket support for streaming
- **Containerization:** Docker + Docker Compose
- **Deployment:** Railway, Render (free tier) or AWS/GCP

Frontend (Optional):

- **Streamlit:** Rapid prototyping
- **Gradio:** ML model demos
- **React + FastAPI:** Production-grade UI

▮ Implementation Steps

Phase 1: Foundation (Week 1) - 15-20 hours

Step 1.1: Environment Setup

```
# Create project structure
mkdir multi-agent-research
cd multi-agent-research

# Initialize Python environment
uv init # or poetry init
uv venv
source .venv/bin/activate

# Install dependencies
uv add langgraph langchain-openai langchain-community
uv add tavily-python # Web search
uv add chromadb # Vector DB (embedded option)
uv add trulens-eval # Evaluation
uv add fastapi uvicorn # API
uv add python-dotenv # Environment variables
```

Step 1.2: Project Structure

```
multi-agent-research/
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── planner.py
│   │   ├── researcher.py
│   │   ├── analyst.py
│   │   ├── synthesizer.py
│   │   └── critic.py
│   ├── tools/
│   │   ├── __init__.py
│   │   ├── web_search.py
│   │   ├── vector_search.py
│   │   ├── python_repl.py
│   │   └── web_scraper.py
│   ├── graph/
│   │   ├── __init__.py
│   │   ├── state.py
│   │   └── workflow.py
│   ├── evaluation/
│   │   ├── __init__.py
│   │   ├── rag_triad.py
│   │   └── custom_metrics.py
│   ├── api/
│   │   ├── __init__.py
│   │   └── main.py
│   └── utils/
│       └── __init__.py
```

```

├── logger.py
├── config.py
├── tests/
├── notebooks/
├── config/
│   └── config.yaml
├── .env.example
├── pyproject.toml
├── Dockerfile
├── docker-compose.yml
└── README.md

```

Step 1.3: Define State Schema

```

# src/graph/state.py
from typing import List, Dict, Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    """Shared state across all agents"""

    # User input
    query: str

    # Messages (for LangGraph compatibility)
    messages: Annotated[List, add_messages]

    # Planner outputs
    research_plan: List[Dict] # List of tasks

    # Research Agent outputs
    web_results: List[Dict] # Web search findings
    documents: List[Dict] # Retrieved documents
    scraped_content: List[Dict] # Scraped URLs

    # Analyst outputs
    analysis_results: Dict # Data analysis findings
    code_snippets: List[str] # Python code executed

    # Synthesis outputs
    draft_report: str

    # Critic outputs
    quality_assessment: Dict
    revision_requests: List[str]
    approved: bool

    # Metadata
    iteration_count: int
    agent_interactions: List[Dict] # Log of agent actions

    # Final outputs
    final_report: str

```



```
citations: List[Dict]
metrics: Dict
```

Phase 2: Agent Implementation (Week 2) - 20-25 hours

Step 2.1: Implement Planner Agent

```
# src/agents/planner.py
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate

PLANNER_PROMPT = """You are a research planning expert.
Given a complex research question, break it down into specific sub-tasks.

Research Question: {query}

Create a structured research plan with:
1. Information gathering tasks (web search queries)
2. Data analysis tasks (if quantitative analysis needed)
3. Synthesis requirements

Output as JSON:
{{
  "tasks": [
    {{
      "type": "web_search",
      "query": "specific search query",
      "priority": 1,
      "agent": "researcher"
    }},
    {{
      "type": "data_analysis",
      "description": "what to analyze",
      "priority": 2,
      "agent": "analyst"
    }}
  ]
}}
"""

def create_planner_agent():
    llm = ChatOpenAI(model="gpt-4o", temperature=0)
    prompt = ChatPromptTemplate.from_template(PLANNER_PROMPT)
    return prompt | llm

async def planner_node(state: AgentState):
    """Planner agent node"""
    planner = create_planner_agent()

    response = await planner.ainvoke({"query": state["query"]})

    # Parse JSON response
    import json
    plan = json.loads(response.content)
```

```

return {
    "research_plan": plan["tasks"],
    "agent_interactions": state["agent_interactions"] + [{
        "agent": "planner",
        "action": "created_plan",
        "output": plan
    }]
}

```

Step 2.2: Implement Research Agent with Tools

```

# src/tools/web_search.py
from tavily import TavilyClient
import os

def create_web_search_tool():
    tavily_client = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))

    def search_web(query: str, max_results: int = 5) -> List[Dict]:
        """Search the web for information"""
        response = tavily_client.search(
            query=query,
            max_results=max_results,
            include_answer=True,
            include_raw_content=True
        )

        return [
            {
                "url": result["url"],
                "title": result["title"],
                "content": result["content"],
                "score": result["score"]
            }
            for result in response["results"]
        ]

    return search_web

# src/agents/researcher.py
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from src.tools.web_search import create_web_search_tool
from src.tools.vector_search import create_vector_search_tool

RESEARCHER_PROMPT = """You are an expert research agent.
Your task: {task_description}

Use available tools to gather comprehensive information:
- web_search: For current information from the internet
- vector_search: For relevant documents in knowledge base

Guidelines:

```

1. Search multiple angles of the topic
2. Verify information from multiple sources
3. Note the URL/source for every piece of information
4. Summarize key findings

Provide findings in this format:

```
## Finding: [Brief title]
Content: [Detailed finding]
Source: [URL or document]
Confidence: [High/Medium/Low]
"""
```

```
def create_researcher_agent():
    llm = ChatOpenAI(model="gpt-4o", temperature=0)

    tools = [
        create_web_search_tool(),
        create_vector_search_tool()
    ]

    prompt = ChatPromptTemplate.from_messages([
        ("system", RESEARCHER_PROMPT),
        ("placeholder", "{chat_history}"),
        ("human", "{input}"),
        ("placeholder", "{agent_scratchpad}")
    ])

    agent = create_tool_calling_agent(llm, tools, prompt)
    agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

    return agent_executor

async def researcher_node(state: AgentState):
    """Research agent node"""
    researcher = create_researcher_agent()

    # Execute research tasks from plan
    research_tasks = [
        task for task in state["research_plan"]
        if task["type"] == "web_search"
    ]

    all_results = []
    for task in research_tasks[:3]: # Limit to prevent infinite loops
        result = await researcher.ainvoke({
            "input": task["query"],
            "chat_history": []
        })
        all_results.append(result)

    return {
        "web_results": all_results,
        "agent_interactions": state["agent_interactions"] + [{
            "agent": "researcher",
            "action": "web_search",
            "tasks_completed": len(research_tasks)
        }]
```

```
}]
}
```

Step 2.3: Implement Data Analyst Agent

```
# src/agents/analyst.py
from langchain_experimental.tools import PythonREPLTool
from langchain.agents import create_tool_calling_agent, AgentExecutor

ANALYST_PROMPT = """You are a data analysis expert.
Research findings: {research_findings}

Your task: {analysis_task}

Use Python REPL to:
1. Process and analyze data
2. Perform statistical calculations
3. Create data summaries

Important:
- Write safe, executable Python code
- Use pandas, numpy for data processing
- Provide clear explanations of findings
- Include confidence intervals where applicable

Output format:
### Analysis: [Title]
Method: [Statistical method used]
Code: ```python [code] ```
Result: [Numerical result]
Interpretation: [What it means]
"""

def create_analyst_agent():
    llm = ChatOpenAI(model="gpt-4o", temperature=0)

    tools = [PythonREPLTool()]

    prompt = ChatPromptTemplate.from_messages([
        ("system", ANALYST_PROMPT),
        ("human", "{input}"),
        ("placeholder", "{agent_scratchpad}")
    ])

    agent = create_tool_calling_agent(llm, tools, prompt)
    agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

    return agent_executor

async def analyst_node(state: AgentState):
    """Data analyst agent node"""
    # Only run if there are analysis tasks
    analysis_tasks = [
        task for task in state["research_plan"]
        if task["type"] == "data_analysis"
    ]
```

```

]

if not analysis_tasks:
    return {"analysis_results": {}}

analyst = create_analyst_agent()

results = {}
for task in analysis_tasks:
    result = await analyst.ainvoke({
        "input": task["description"],
        "research_findings": state["web_results"]
    })
    results[task["description"]] = result

return {
    "analysis_results": results,
    "agent_interactions": state["agent_interactions"] + [{
        "agent": "analyst",
        "action": "data_analysis",
        "tasks_completed": len(analysis_tasks)
    }]
}

```

Step 2.4: Implement Synthesis Agent

```

# src/agents/synthesizer.py
SYNTHESIS_PROMPT = """You are an expert research writer.

Research Question: {query}

Web Research Findings:
{web_results}

Data Analysis Results:
{analysis_results}

Your task: Synthesize all findings into a comprehensive, well-structured report.

Requirements:
1. Executive summary (2-3 sentences)
2. Detailed findings organized by topic
3. CITE EVERY CLAIM with [Source: URL] or [Document: Name]
4. Include data analysis results with interpretations
5. Draw conclusions and implications
6. Professional, objective tone

Format the report in Markdown with clear headings.
"""

async def synthesizer_node(state: AgentState):
    """Synthesis agent - combines all findings"""
    llm = ChatOpenAI(model="gpt-4o", temperature=0.3)

    prompt = ChatPromptTemplate.from_template(SYNTHESIS_PROMPT)

```

```

chain = prompt | llm

draft = await chain.ainvoke({
    "query": state["query"],
    "web_results": state["web_results"],
    "analysis_results": state["analysis_results"]
})

return {
    "draft_report": draft.content,
    "agent_interactions": state["agent_interactions"] + [{
        "agent": "synthesizer",
        "action": "draft_created",
        "length": len(draft.content)
    }]
}

```

Step 2.5: Implement Critic Agent

```

# src/agents/critic.py
CRITIC_PROMPT = """You are a critical quality reviewer for research reports.

Original Query: {query}

Draft Report:
{draft_report}

Source Materials:
{web_results}

Your task: Review the draft for:
1. Hallucinations - Are all claims supported by sources?
2. Citation accuracy - Does every claim have a proper citation?
3. Logical consistency - Are arguments coherent?
4. Completeness - Does it address the original query?
5. Factual accuracy - Cross-check claims against sources

Output JSON:
{{
    "approved": true/false,
    "groundedness_score": 0.0-1.0,
    "citation_coverage": 0.0-1.0,
    "issues": [
        {{
            "type": "hallucination/missing_citation/logic_error",
            "description": "specific issue",
            "location": "paragraph or section",
            "severity": "high/medium/low"
        }}
    ],
    "revision_requests": ["specific request 1", "request 2"]
}}
"""

async def critic_node(state: AgentState):

```

```

"""Critic agent - quality control"""
llm = ChatOpenAI(model="gpt-4o", temperature=0)

prompt = ChatPromptTemplate.from_template(CRITIC_PROMPT)
chain = prompt | llm

assessment = await chain.ainvoke({
    "query": state["query"],
    "draft_report": state["draft_report"],
    "web_results": state["web_results"]
})

import json
quality = json.loads(assessment.content)

return {
    "quality_assessment": quality,
    "approved": quality["approved"],
    "revision_requests": quality.get("revision_requests", []),
    "agent_interactions": state["agent_interactions"] + [{
        "agent": "critic",
        "action": "quality_review",
        "approved": quality["approved"]
    }]
}

```

Phase 3: Workflow Orchestration (Week 2-3) - 15-20 hours

Step 3.1: Build LangGraph Workflow

```

# src/graph/workflow.py
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.postgres import PostgresSaver
from src.graph.state import AgentState
from src.agents.planner import planner_node
from src.agents.researcher import researcher_node
from src.agents.analyst import analyst_node
from src.agents.synthesizer import synthesizer_node
from src.agents.critic import critic_node

def create_research_graph():
    """Create the multi-agent research workflow"""

    # Initialize graph
    workflow = StateGraph(AgentState)

    # Add nodes
    workflow.add_node("planner", planner_node)
    workflow.add_node("researcher", researcher_node)
    workflow.add_node("analyst", analyst_node)
    workflow.add_node("synthesizer", synthesizer_node)
    workflow.add_node("critic", critic_node)

    # Define edges

```

```

workflow.set_entry_point("planner")
workflow.add_edge("planner", "researcher")
workflow.add_edge("researcher", "analyst")
workflow.add_edge("analyst", "synthesizer")
workflow.add_edge("synthesizer", "critic")

# Conditional edge from critic
def should_revise(state: AgentState) -> str:
    """Decide if revision is needed"""
    if state["approved"]:
        return "end"
    elif state["iteration_count"] >= 3:
        return "end" # Max 3 iterations
    else:
        return "synthesizer" # Send back for revision

workflow.add_conditional_edges(
    "critic",
    should_revise,
    {
        "synthesizer": "synthesizer",
        "end": END
    }
)

# Add checkpointing for state persistence
checkpointer = PostgresSaver.from_conn_string(
    os.getenv("POSTGRES_URL")
)

return workflow.compile(checkpointer=checkpointer)

# Usage
graph = create_research_graph()

async def run_research(query: str):
    """Run research workflow"""
    initial_state = {
        "query": query,
        "messages": [],
        "iteration_count": 0,
        "agent_interactions": [],
        "approved": False
    }

    final_state = await graph.ainvoke(
        initial_state,
        config={"configurable": {"thread_id": "research-1"}}
    )

    return final_state

```

Step 3.2: Add Error Handling & Retry Logic


```

# src/graph/workflow.py (enhanced)
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=2, max=10)
)
async def researcher_node_with_retry(state: AgentState):
    """Research agent with automatic retry on failure"""
    try:
        return await researcher_node(state)
    except Exception as e:
        logger.error(f"Research agent failed: {e}")
        # Return partial results if available
        return {
            "web_results": [],
            "error": str(e)
        }

# Add timeout protection
import asyncio

async def run_with_timeout(coroutine, timeout=300):
    """Run coroutine with timeout (5 minutes default)"""
    try:
        return await asyncio.wait_for(coroutine, timeout=timeout)
    except asyncio.TimeoutError:
        logger.error("Research workflow timed out")
        raise

```

Phase 4: Evaluation System (Week 3) - 10-15 hours

Step 4.1: Implement RAG Triad Evaluation

```

# src/evaluation/rag_triad.py
from trulens_eval import TruChain, Feedback, Tru
from trulens_eval.feedback import Groundedness
from trulens_eval.feedback.provider.openai import OpenAI as OpenAIProvider

def setup_evaluation():
    """Setup TruLens evaluation"""

    tru = Tru()
    tru.reset_database()

    provider = OpenAIProvider()

    # Context Relevance
    f_context_relevance = (
        Feedback(provider.context_relevance)
        .on_input_output()
        .aggregate(np.mean)
    )

```

```

# Groundedness
grounded = Groundedness(groundedness_provider=provider)
f_groundness = (
    Feedback(grounded.groundedness_measure_with_cot_reasons)
    .on_output()
    .aggregate(grounded.grounded_statements_aggregator)
)

# Answer Relevance
f_answer_relevance = (
    Feedback(provider.relevance)
    .on_input_output()
)

return [f_context_relevance, f_groundness, f_answer_relevance]

async def evaluate_research_output(query: str, response: str, sources: List[Dict]):
    """Evaluate research quality"""

    feedbacks = setup_evaluation()

    # Create evaluation record
    from trulens_eval import TruChain

    tru_recorder = TruChain(
        chain_callable, # Your research function
        app_id="multi_agent_research",
        feedbacks=feedbacks
    )

    with tru_recorder as recording:
        result = await run_research(query)

    # Get metrics
    records = tru_recorder.get_records()[^0]

    return {
        "context_relevance": records["context_relevance"],
        "groundedness": records["groundedness"],
        "answer_relevance": records["answer_relevance"]
    }

```

Step 4.2: Custom Evaluation Metrics

```

# src/evaluation/custom_metrics.py
import re
from typing import List, Dict

def evaluate_citation_coverage(report: str, sources: List[Dict]) -> float:
    """Calculate % of statements with citations"""

    # Split into sentences
    sentences = re.split(r'[.!?]+', report)

```

```

# Count sentences with citations
citation_pattern = r'\[Source:|Document:|URL:'
cited_sentences = [
    s for s in sentences
    if re.search(citation_pattern, s)
]

coverage = len(cited_sentences) / len(sentences) if sentences else 0
return round(coverage, 2)

def evaluate_task_success(state: AgentState) -> bool:
    """Did the workflow complete successfully?"""
    return (
        state.get("approved", False) and
        len(state.get("final_report", "")) > 500 and
        len(state.get("citations", [])) > 0
    )

def calculate_agent_efficiency(state: AgentState) -> Dict:
    """Measure agent collaboration efficiency"""
    interactions = state.get("agent_interactions", [])

    return {
        "total_interactions": len(interactions),
        "planner_calls": len([i for i in interactions if i["agent"] == "planner"]),
        "researcher_calls": len([i for i in interactions if i["agent"] == "researcher"]),
        "iterations": state.get("iteration_count", 0)
    }

def calculate_token_usage(state: AgentState) -> Dict:
    """Estimate token usage and cost"""
    # Implementation depends on your LLM provider's usage tracking
    # Placeholder example
    return {
        "input_tokens": 5000,
        "output_tokens": 3000,
        "estimated_cost_usd": 0.15
    }

```

Phase 5: API & Deployment (Week 4) - 15-20 hours

Step 5.1: FastAPI with Streaming

```

# src/api/main.py
from fastapi import FastAPI, WebSocket
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
import asyncio
import json

app = FastAPI(title="Multi-Agent Research API")

class ResearchRequest(BaseModel):
    query: str

```

```

    max_iterations: int = 3
    enable_streaming: bool = False

class ResearchResponse(BaseModel):
    query: str
    report: str
    citations: List[Dict]
    metrics: Dict
    agent_log: List[Dict]

@app.post("/research", response_model=ResearchResponse)
async def conduct_research(request: ResearchRequest):
    """Conduct research and return complete report"""

    graph = create_research_graph()

    initial_state = {
        "query": request.query,
        "messages": [],
        "iteration_count": 0,
        "agent_interactions": [],
        "approved": False
    }

    final_state = await graph.ainvoke(initial_state)

    # Evaluate quality
    metrics = await evaluate_research_output(
        request.query,
        final_state["final_report"],
        final_state["web_results"]
    )

    return ResearchResponse(
        query=request.query,
        report=final_state["final_report"],
        citations=final_state.get("citations", []),
        metrics=metrics,
        agent_log=final_state["agent_interactions"]
    )

@app.websocket("/ws/research")
async def research_websocket(websocket: WebSocket):
    """Streaming research with real-time updates"""
    await websocket.accept()

    try:
        # Receive query
        data = await websocket.receive_json()
        query = data["query"]

        graph = create_research_graph()

        # Stream updates
        async for event in graph.astream({
            "query": query,

```

```

        "messages": [],
        "iteration_count": 0,
        "agent_interactions": [],
        "approved": False
    }):
        # Send agent updates to client
        await websocket.send_json({
            "type": "agent_update",
            "data": event
        })

        # Send final result
        await websocket.send_json({
            "type": "complete",
            "data": event # Final state
        })

    except Exception as e:
        await websocket.send_json({
            "type": "error",
            "message": str(e)
        })
    finally:
        await websocket.close()

@app.get("/health")
async def health_check():
    """Health check endpoint"""
    return {"status": "healthy"}

```

Step 5.2: Docker Deployment

```

# Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Copy dependency files
COPY pyproject.toml uv.lock ./

# Install dependencies
RUN pip install uv
RUN uv sync --frozen

# Copy application
COPY src/ ./src/

# Expose port
EXPOSE 8000

```

```
# Run application
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# docker-compose.yml
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - TAVILY_API_KEY=${TAVILY_API_KEY}
      - POSTGRES_URL=postgresql://user:pass@postgres:5432/research
    depends_on:
      - postgres
      - redis
    volumes:
      - ./src:/app/src

  postgres:
    image: postgres:15
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=research
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  postgres_data:
```

▮ Evaluation Criteria & Success Metrics

Minimum Viable Product (MVP) Criteria

Functional Requirements (Must Have):

- ✔ 4+ agents collaborating (Planner, Researcher, Analyst, Synthesizer, Critic)
- ✔ At least 2 tools integrated (web search + Python REPL)
- ✔ State persistence with checkpointing
- ✔ Quality evaluation with at least RAG Triad metrics
- ✔ Generates reports >1000 words with citations

- ✓ API deployment (FastAPI)

Quality Metrics (Target Performance):

- ✓ Context Relevance: >0.80
- ✓ Groundedness: >0.90
- ✓ Answer Relevance: >0.85
- ✓ Citation Coverage: >70% of claims cited
- ✓ Task Success Rate: >80% of test queries
- ✓ End-to-end latency: <2 minutes

Engineering Standards:

- ✓ Modular code structure (separate files for each agent)
- ✓ Error handling and retry logic
- ✓ Logging throughout workflow
- ✓ Docker containerization
- ✓ Environment variable configuration
- ✓ README with architecture diagram

Example Queries for Testing

Test Cases (Varying Difficulty)

Level 1: Simple Factual Query

"What are the key features of Python 3.12 released in 2024?"

Expected: Web search, summary of features, official docs citations

Level 2: Comparative Analysis

"Compare the carbon footprint of electric vehicles vs. hybrid vehicles in 2026"

Expected: Multi-source research, data analysis, comparison table

Level 3: Complex Multi-Domain

"Analyze the impact of Indonesia's new EV policies on automotive manufacturing, consumer

Expected: Multiple sub-tasks, web research, data analysis, synthesis across domains

Level 4: Time-Sensitive Data

"What are the current top 3 AI startups by funding in Southeast Asia, and what are they k

Expected: Real-time web search, data extraction, verification

Level 5: Requires Reasoning

"If palm oil production in Malaysia increases by 15% while Indonesia maintains current le

Expected: Web research for current data, Python analysis for projections, reasoning

▮ Stretch Goals (Advanced Features)

Optional Enhancements (After MVP)

1. GraphRAG Integration (+15 hours)

- Build knowledge graph from research findings
- Use Neo4j for entity relationships
- Enhanced retrieval with graph traversal

2. Human-in-the-Loop Checkpoints (+10 hours)

- Pause workflow for human review
- Approve/reject agent decisions
- Provide feedback to agents

3. Multi-Document Processing (+10 hours)

- Upload PDF documents for analysis
- Extract and index content
- Cross-reference with web research

4. Interactive Dashboard (+15 hours)

- Streamlit/React frontend
- Real-time agent activity visualization
- Metrics dashboard
- Report export (PDF, DOCX)

5. Cost Optimization (+8 hours)

- Smart LLM routing (GPT-4 vs GPT-4o-mini)
- Caching layer for repeated queries
- Token usage tracking and budgeting

6. Multi-Language Support (+12 hours)

- Accept queries in Bahasa Indonesia
- Conduct research in multiple languages
- Translate final report

▮ Documentation Requirements

README.md **Must Include**

1. Architecture Diagram (Mermaid or image)

- Show agent flow
- Tool integrations
- State management

2. Project Description

- What problem it solves
- How it works (high-level)
- Key differentiators

3. Quick Start

```
# Clone
git clone [repo]

# Setup
docker-compose up

# Test
curl -X POST http://localhost:8000/research \
  -H "Content-Type: application/json" \
  -d '{"query": "What is LangGraph?"}'
```

4. Example Output

- Screenshot or markdown of sample research report
- Metrics dashboard screenshot

5. Evaluation Results

- Table of test queries with metrics
- Performance benchmarks

6. Technical Details

- Agent descriptions
- Tool explanations
- Evaluation methodology

7. Future Improvements

- Known limitations
- Planned enhancements

▮ Estimated Time Breakdown

Phase	Tasks	Hours
Week 1: Foundation	Setup, state schema, tools	15-20
Week 2: Agents	5 agents + tool integration	20-25
Week 2-3: Workflow	LangGraph orchestration, error handling	15-20
Week 3: Evaluation	RAG Triad + custom metrics	10-15
Week 4: Deployment	FastAPI, Docker, testing	15-20
Total	MVP Complete	75-100 hours

Part-time schedule (20 hrs/week): **4-5 weeks**

Full-time sprint (40 hrs/week): **2-2.5 weeks**

▮ Portfolio Impact Assessment

Hiring Impact Score: **85/100** ★★★

Why This Project Stands Out:

1. **Cutting-Edge Tech:** Agentic AI is THE most in-demand skill for AI Engineers in 2026
2. **Production Engineering:** State management, error handling, deployment
3. **Evaluation Focus:** Shows you understand non-deterministic system quality
4. **Systems Thinking:** Multi-component orchestration
5. **Business Relevance:** Solves real research automation problem

Target Roles:

- AI Engineer (primary fit)
- ML Engineer (shows architecture skills)
- Applied AI Researcher

Salary Impact: Can justify \$15-25K higher salary expectations for demonstrated agentic AI expertise

▮ Required Resources

API Keys Needed (Budget: ~\$30-50 for development):

- OpenAI API (\$20 credit sufficient for testing)
- Tavily API (1000 free searches/month, then \$0.003/search)
- Pinecone (free tier: 1 index, 100K vectors)

Learning Resources:

- LangGraph Documentation: <https://python.langchain.com/docs/langgraph>
- TruLens Guide: <https://www.trulens.org/>
- Multi-Agent Patterns: <https://www.deeplearning.ai/short-courses/ai-agents-in-langgraph/>

Community Support:

- LangChain Discord
- r/LangChain subreddit
- LangGraph GitHub discussions

✓ Definition of Done

Project is complete when:

- [] All 5 agents implemented and working
- [] State persistence with PostgreSQL/Redis
- [] RAG Triad evaluation showing metrics >target
- [] FastAPI deployed and accessible
- [] Docker Compose setup working
- [] README with architecture diagram
- [] 5+ test queries demonstrating success
- [] Blog post written explaining the system
- [] Code pushed to GitHub with proper documentation
- [] Demo video (3-5 minutes) recorded

This is your flagship AI Engineering project. Take the time to build it right, document thoroughly, and it will open doors to senior AI Engineer roles paying \$120K-200K+ in 2026.

Good luck! ▮



